

I-JEPA

Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture

Assran, Duval, Misra, Bojanowski, Vincent, Rabbat, LeCun, Ballas

Meta AI (FAIR) · McGill University · Mila · NYU | arXiv:2301.08243 · 2023

Background on Masked Autoencoders (MAE)

Self-Supervised Learning

Goal: Learn rich image representations without manual labels

Invariance-Based Methods

- Optimize encoder to produce similar embeddings for two views of the same image
- Views constructed via hand-crafted augmentations (crop, jitter, flip)
- High semantic representations but introduces strong augmentation biases
- Difficult to generalize across modalities (e.g., audio)
- Examples: SimCLR, DINO, iBOT, BYOL, MSN

Generative / Reconstruction Methods

- Remove or corrupt portions of the input, then learn to reconstruct
- Mask-denoising reconstructs masked patches at pixel or token level
- Less augmentation knowledge required; generalizes to other modalities
- Often lower semantic level — biased toward pixel details
- Examples: MAE, BEiT, SimMIM, data2vec, Context Encoders

I-JEPA introduces a third path: JEPA — predict in representation space without hand-crafted augmentations

Three SSL Architecture Families

All can be described using the Energy-Based Model (EBM) framework

Joint-Embedding Architecture (JEA)

$x \rightarrow \text{Encoder}$

$y \rightarrow \text{Encoder}$

Minimize $D(s_x, s_y)$

- Outputs similar embeddings for compatible x, y inputs
- Risk: representation collapse (constant embeddings)
- Needs contrastive loss or asymmetric design
- High semantic level but Augmentation-dependent

Generative Architecture

$x \rightarrow \text{Encoder}$

$z + \text{Decoder}$

Reconstruct y (pixels)

- Directly reconstruct input from masked/corrupted version
- No collapse risk (low-capacity conditioning)
- Predicts in pixel / token space

Joint-Embedding Predictive Arch. (JEPA)

$x \rightarrow \text{Encoder}$

Predictor + z

Predict s_y in embed. space

- Predict embeddings of y from x via predictor network
- Loss in embedding space $\rightarrow$ abstract, semantic targets
- Avoids pixel-level detail & augmentation bias

Masked Autoencoders (MAE)

He et al., 2022 | Meta AI + UC Berkeley | CVPR 2022

Masked Autoencoders (MAE): Core Idea

A scalable self-supervised approach for vision, mask most of the image, reconstruct the missing patches

75%

of patches
are masked

196

total patches
(16×16 grid)

Pixel

reconstruction
target

L2

loss in
pixel space

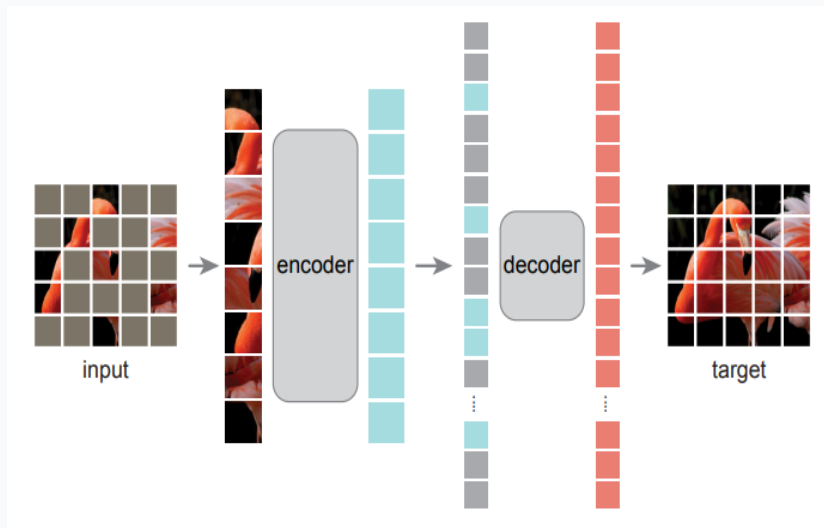
Key Properties

- Asymmetric encoder-decoder: encoder processes only visible (unmasked) patches. Decoder is lightweight and only used during pretraining
- Random sampling with a high masking ratio largely eliminates redundancy, thus creating a task that cannot be easily solved by extrapolation from visible neighbouring patches
- Pixel reconstruction: forces model to understand low-level structure
- Strong performance after fine-tuning; weaker at linear probing (semantic representations)

MAE: Architecture & Training Pipeline

Pixel Recon.

Reconstruct
masked patch
pixel values
MSE loss



Reconstruction target. Our MAE reconstructs the input by predicting the *pixel* values for each masked patch. Each element in the decoder's output is a vector of pixel values representing a patch. The last layer of the decoder is a linear projection whose number of output channels equals the number of pixel values in a patch. The decoder's output is reshaped to form a reconstructed image. Our loss function computes the mean squared error (MSE) between the recon-

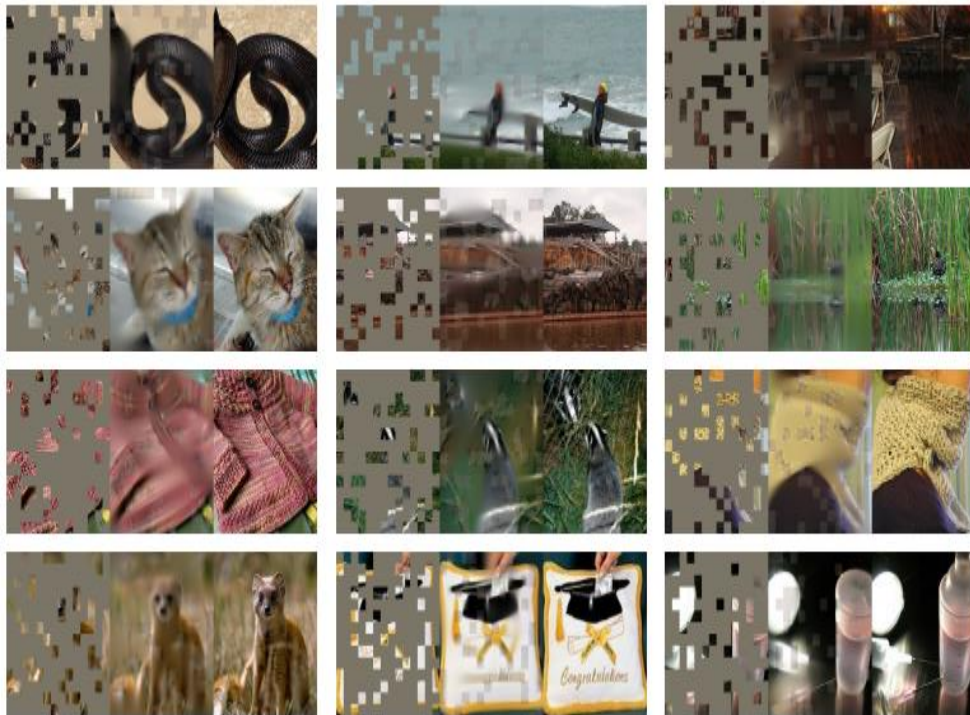
MAE Limitations vs I-JEPA

Pixel targets → low-level features, poor linear probing

Requires 1600 epochs; slow to converge

10× more compute than I-JEPA for ViT-H/14

MAE: Architecture & Training Pipeline



Left image is the masked image

Middle image is the reconstructed output from the masked autoencoder.

Right image is the ground truth.

75% mask is used.

MAE: Results & Why I-JEPA Does Better

Method	Arch.	Epochs	Linear Probe Top-1	1% Semi-Sup.
MAE	ViT-B/16	1600	68.0%	N/A
MAE	ViT-L/16	1600	76.0%	67.1%
MAE	ViT-H/14	1600	77.2%	71.5%
I-JEPA	ViT-H/14	300	79.3% (+2.1%!)	73.3% (+1.8%!)

Root Cause of the Gap

MAE predicts in pixel space: irrelevant low-level detail (texture, exact colors) drowns out semantic content.

The target encoder in I-JEPA produces abstract representations, pixel noise is filtered before the loss is computed.

I-JEPA Advantages

- 5× fewer training epochs (300 vs 1600)
- 10× less compute for ViT-H/14
- Higher linear-probe accuracy (semantic)
- Better transfer on CIFAR-100, Places205
- No hand-crafted augmentations needed

I-JEPA Architecture & Method

Image-based Joint-Embedding Predictive Architecture

I-JEPA: The Core Idea

"Given a single context block, predict the representations of various target blocks in the same image."

Predict in Representation Space

Unlike MAE which predicts pixels, I-JEPA predicts abstract embeddings from a learned target-encoder. Pixel-level noise is eliminated before the loss is applied, guiding the model to learn semantic features.

Multi-Block Masking Strategy

Target blocks are sampled at a semantic scale (The context block is spatially distributed large regions and informative). This forces the model to learn high-level understanding rather than local textures.

Exponential Moving Average Target

A target encoder (frozen copy updated via EMA) produces stable, abstract prediction targets. The context encoder learns by trying to predict these targets.

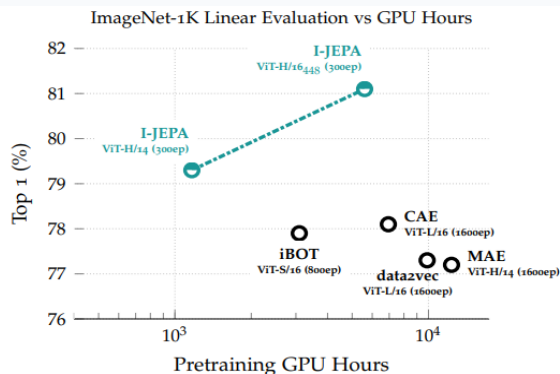


Figure 1. **ImageNet Linear Evaluation.** The I-JEPA method learns semantic image representations without using any view data augmentations during pretraining. By predicting in representation space, I-JEPA produces semantic representations while using less compute than previous methods.

I-JEPA: Architecture

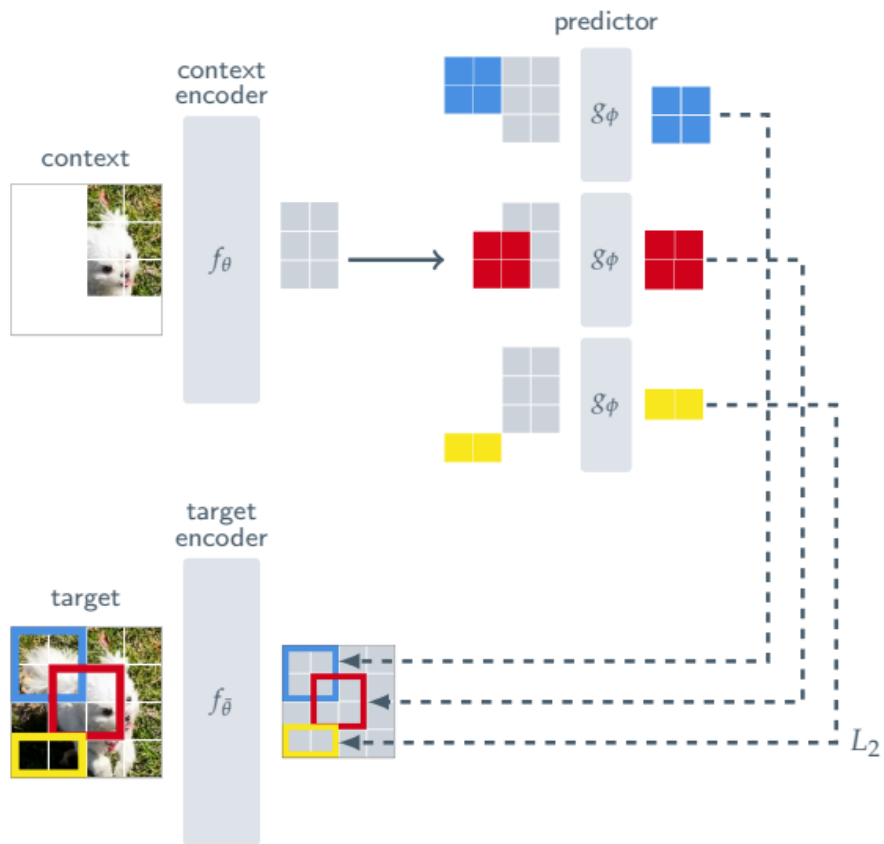


Figure 3. **I-JEPA.** The Image-based Joint-Embedding Predictive Architecture uses a single context block to predict the representations of various target blocks originating from the same image. The context encoder is a Vision Transformer (ViT), which only processes the visible context patches. The predictor is a narrow ViT that takes the context encoder output and, conditioned on positional tokens (shown in color), predicts the representations of a target block at a specific location. The target representations correspond to the outputs of the target-encoder, the weights of which are updated at each iteration via an exponential moving average of the context encoder weights.

I-JEPA: Architecture Components

Context Encoder (f_{θ})

- Standard ViT architecture (B/L/H size)
- Processes only visible (unmasked) context patches
- Weights learned via gradient descent
- Produces patch-level representations s_x
- Context block: scale (0.85, 1.0), unit aspect ratio

Target Encoder ($\tilde{f}_{\theta}$)

- Same ViT architecture as context encoder
- Weights are NOT directly trained
- Updated via exponential moving average (EMA) of context encoder
- Momentum: 0.996 $\rightarrow$ 1.0 during training
- Produces abstract, stable target representations s_y

Predictor (g_{ϕ})

- Narrow (lightweight) ViT: 384-dim embedding
- Conditioned on positional mask tokens
- Takes context encoder output s_x as input
- Applied M times (one per target block)
- Outputs predicted representations $\hat{s}_y(i)$

Loss: Average L_2 distance between predicted $\hat{s}_y(i)$ and target $s_y(i)$ representations across M target blocks

I-JEPA: Implementation Details

Prediction. Given the output of the context encoder, s_x , we wish to predict the M target block representations $s_y(1), \dots, s_y(M)$. To that end, for a given target block $s_y(i)$ corresponding to a target mask B_i , the predictor $g_\phi(\cdot, \cdot)$ takes as input the output of the context encoder s_x and a mask token for each patch we wish to predict, $\{m_j\}_{j \in B_i}$, and outputs a patch-level prediction $\hat{s}_y(i) = \{\hat{s}_{y_j}\}_{j \in B_i} = g_\phi(s_x, \{m_j\}_{j \in B_i})$. The mask tokens are

parameterized by a shared learnable vector with an added positional embedding. Since we wish to make predictions for M target blocks, we apply our predictor M times, each time conditioning on the mask tokens corresponding to the target-block locations we wish to predict, and obtain predictions $\hat{s}_y(1), \dots, \hat{s}_y(M)$.

Loss. The loss is simply the average L_2 distance between the predicted patch-level representations $\hat{s}_y(i)$ and the target patch-level representation $s_y(i)$; i.e.,

$$\frac{1}{M} \sum_{i=1}^M D(\hat{s}_y(i), s_y(i)) = \frac{1}{M} \sum_{i=1}^M \sum_{j \in B_i} \|\hat{s}_{y_j} - s_{y_j}\|_2^2.$$

The parameters of the predictor, ϕ , and context encoder, θ , are learned through gradient-based optimization, while the parameters of the target encoder $\bar{\theta}$ are updated via an exponential moving average of the context-encoder parameters.

I-JEPA doesn't explicitly use a latent variable "z".

All M targets are predicted from the same context!

M=4 used in the paper.

Multi-Block Masking Strategy

The masking strategy is the core design choice that guides I-JEPA to learn semantic representations

Masking Parameters

Target blocks (M=4)

Scale: (0.15, 0.2) of image
Aspect ratio: (0.75, 1.5)
Possibly overlapping each other

Context block (M=1)

Scale: (0.85, 1.0) of image
Unit aspect ratio
Removed: overlapping target regions

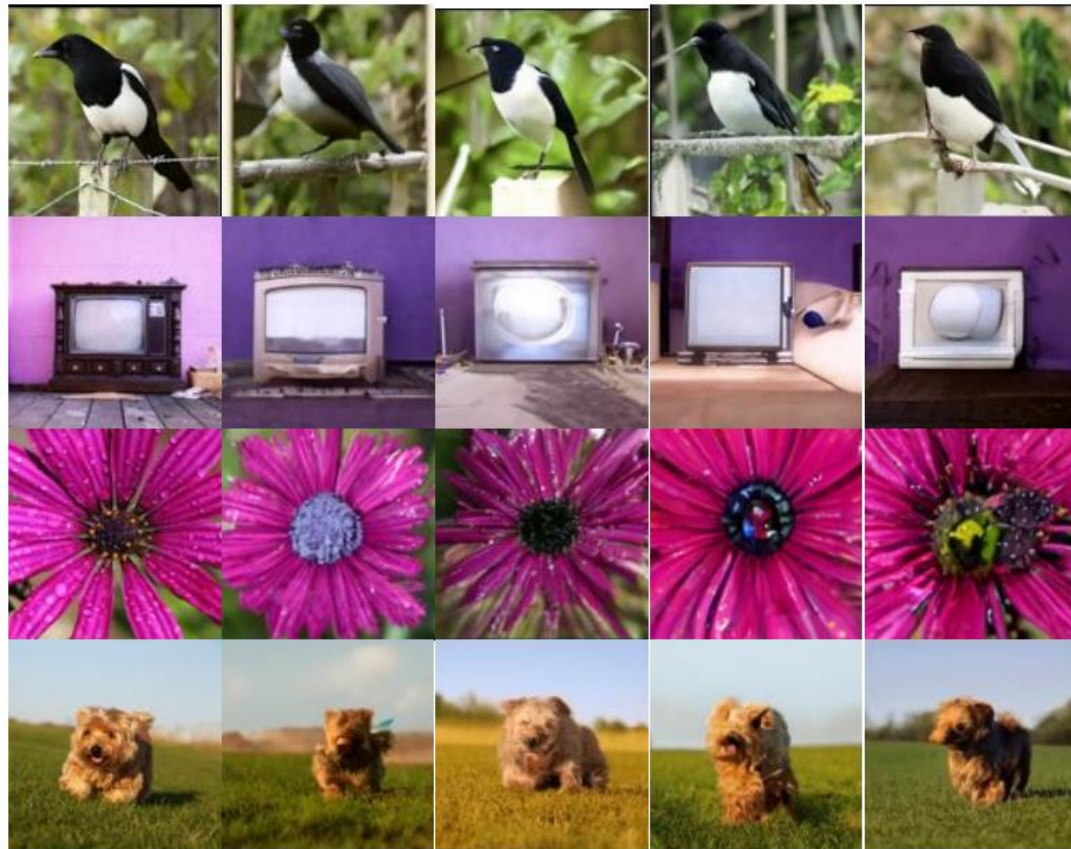
Key: Target masking applied to TARGET-ENCODER OUTPUT, not the input ensures semantic-level targets

Masking Ablation (Table 6 — 1% ImageNet)

Mask Strategy	Top-1 (1% labels)
★ multi-block	54.2%
rasterized	15.5%
block	20.2%
random	17.6%

Multi-block masking outperforms alternatives by 34+ percentage points!

I-JEPA: Visualizing the outputs



First column: original image

Subsequent columns: samples from a generative model decoding the output of an I-JEPA target-encoder.

Qualities that are common across samples represent information that contained in the I-JEPA representation. I-JEPA is able to correctly capture the high-level information regarding objects and their poses.